

Relatório - Scribo

1. Escolha da Stack

- **Linguagem:** php, css, JavaScript, html
- **Framework:** Laravel
- **Banco de dados:** MySQL

Essa combinação é amplamente utilizada no mercado, oferecendo boa escalabilidade, segurança e suporte da comunidade.

2. Bibliotecas para JWT

- php-open-source-saver/jwt-auth
 - Permite geração, validação e personalização de tokens JWT
 - Controle total sobre claims e assinatura
- Alternativa:
 - Laravel Sanctum
 - Mais simples para SPAs, porém não utiliza JWT puro

2. Estratégias de Segurança

- **HTTPS obrigatório**
Pra evitar que alguém intercepte os dados
- **Token com tempo de expiração (exp)**
Tokens não vão durar muito tempo (ex: 15 min)
- **Nada sensível no token**
Não colocar senha ou dados importantes no payload
Só coisas básicas tipo:
 - id do usuário
 - nome
 - permissões
- **Chave secreta protegida**
Vai ficar no .env do Laravel e não no código

- **Armazenamento no front-end**

Evitar LocalStorage

O ideal é usar cookies HttpOnly, porque são mais seguros contra ataque XSS

3. Fluxo de autenticação

1. O usuário envia email e senha para a rota de login da aplicação.
2. O Laravel recebe a requisição e valida as credenciais no banco de dados.
3. Caso os dados estejam corretos:
 - o sistema gera um token JWT, contendo as claims básicas (identificador do usuário e tempo de expiração).
4. O token é retornado ao cliente (ou armazenado automaticamente em cookie HttpOnly).

Uso no sistema

- A cada requisição para rotas protegidas, o token JWT é enviado ao backend:
- O Laravel valida o token, verificando:
 - a assinatura do JWT
 - o tempo de expiração (exp)

Se o token for válido, o acesso à rota protegida é liberado. Caso contrário, a requisição é negada.

Resumo do fluxo

Login → Laravel valida usuário → gera JWT → token é armazenado → enviado nas requisições → Laravel valida → acesso liberado

4. Testes

> Algorithm & Token Type

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

✓ Valid header

Payload

> Data

```
{
  "sub": 1,
  "name": "luisa",
  "role": "user",
  "iat": 1710000000,
  "exp": 1710003600
}
```

✓ Valid payload

Sign JWT

BASE64URL ENCODED ☐

> Secret

minha_chave_super_secreta

✓ Valid secret

> Encoded JWT

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3dWI0fEsImShbWU0Ij5dhlzYSIsInJvbGU0Ij1ic2VyIiwiaWF0IjoxNzEwMDAwMDAwLCJleHA10IjE3MTAwMDM2MD89.6SxqDjBVIffDOPC1GbFLZQg6oSGMdZ6Vb_ZSyokH3fos

[Share feedback](#) | [Report issue](#)

O Header define o tipo (JWT) e o algoritmo (HS256). O Payload contém dados básicos do usuário (id, nome, tipo e expiração), sem informações sensíveis. O Secret é a chave usada para assinar o token. Do lado direito aparece o JWT pronto, formado por três partes: header, payload e assinatura. A assinatura garante que, se alguém alterar o conteúdo, o token deixa de ser válido.